Magnetometer Simulation for Autoresearch

September 15th, 2026

Proton magnetometer background

- Measures magnetic field
- Polarization coil aligns water protons
- Pulse turns off
- Protons precess in Earth’s field
- Sensing coil receives a tiny voltage

Frequency measures field

- Precession frequency depends on field strength
- 50 μT produces about 2.13 kHz
- Frequency error becomes field error
- Small signal errors matter

Why the signal is difficult

- Signal: about microvolts
- Noise: coil, amplifier, supply, interference
- Pulse ringing can hide the signal
- Incorrect frequency gives incorrect field

Auto-research and text optimization

- **Common loop**
  - Agent proposes a change
  - Evaluator measures the result
  - Better changes are kept
- **AlphaEvolve (Novikov et al., 2025)**
  - Agent changes algorithm code
  - Evaluators provide selection feedback
  - Found a faster data-center scheduler and improved 4 × 4 complex matrix multiplication
- **Karpathy autoresearch (Karpathy, 2026)**
  - Agent edits one training file
  - Each trial trains for five minutes
  - Validation bits per byte decides whether to keep the change
- **AutoHarness (Lou et al., 2026)**
  - Agent builds and refines a code harness
  - Environment feedback finds invalid behavior
  - Harness prevented illegal moves in 145 text-game environments

System being optimized

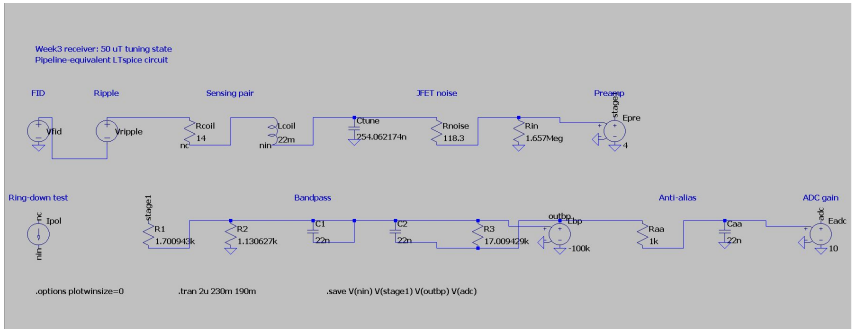
- Receiver after the sensing coils
- Input: coil signal and impedance
- Input: amplifier noise, gain, and filter
- Input: ADC range and resolution
- Input: estimator and clock
- Output: one magnetic-field estimate

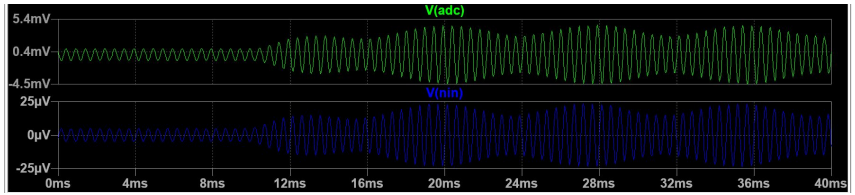
Week 3 coil inputs

- The coil is not optimized during autoresearch
- The following coil parameters are fixed:
  - Polarizer: 360 turns and 3 A
  - Modeled polarizer field: 9.38 mT
  - Two sensing coils: 6 × 6 cm
  - 552 turns per sensing coil
  - Sensing pair: 14 Ω and 22 mH
  - Tuning capacitor: 264 nF
  - Provisional signal decay: 0.95 s

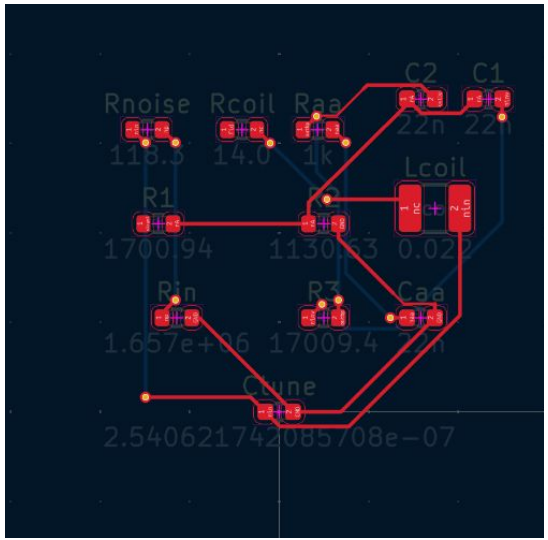
Optimization agent outputs

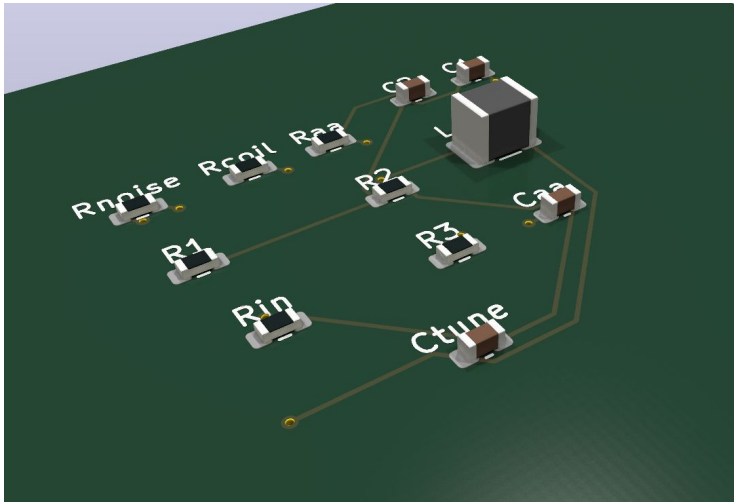
- Circuit proposal files
  - Circuit description: Python .py
  - Simulation netlist: SPICE .cir
  - Simulation results: text tables
  - Schematic: KiCad .kicad_sch
  - Board layout: KiCad .kicad_pcb
- Firmware files
  - Estimator source: C .c and .h
  - Python creates synthetic ADC samples
  - Compiled C code estimates frequency
  - Same estimator is intended for hardware





Skills allow agents to convert pipeline outputs to KiCad PCB schematics





Evaluation

- Simulation step
  - SPICE calculates gain and noise
  - SPICE finds resonance and bandwidth
  - SPICE estimates ring-down
  - Pipeline creates filtered ADC records
  - Records include random noise and ripple
- Estimation step
  - C firmware reads each ADC record
  - Firmware estimates FID frequency
  - Pipeline repeats with random noise
  - Frequency error becomes field error
  - 0.042576 Hz equals 1 nT

How a proposal is graded

- Score: worst field RMS error from 25 to 65 μT
- Lower score is better
- Failed gate gives infinite score
- Only proposals passing the following gates are accepted:
  - ADC clipping: signal must fit the ADC
  - Ring-down: receiver must recover quickly
  - Gross error: estimator must find the FID
  - Rail ripple: supply noise must not corrupt the FID
  - Clock bias: timing error must stay below 0.1 nT
  - Hardware characterization: coil assumptions must be measured

Next steps

- Build and measure the sensing pair
- Measure resistance, inductance, resonance, and Q
- Measure ring-down after a 3 A pulse
- Capture a real FID
- Measure real FID amplitude
- Measure supply ripple and rejection
- Update model and run physically aligned optimization